Clustered Data Simulation with dylpr

Clustered Data Simulation

This document illustrates how to generate clustered data and simulate potential outcomes using the `dplyr` package.

```r
library("dplyr")
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

Build Cluster Function

```r
build_cluster <- function(K = 1000,
                          dist = c("poisson", "nbinom", "uniform", "fixed"),
                          lambda = 10,
                          size = NULL,
                          prob = NULL,
                          min_size = 2,
                          max_size = NULL,
                          fixed_size = NULL,
                          seed = NULL) {
  if (!is.null(seed)) {
    set.seed(seed)
  }

  dist <- match.arg(dist)

  if (!is.numeric(K) || length(K) != 1 || K <= 0) {
    stop("K must be a single positive number.")
  }
  if (!is.numeric(min_size) || length(min_size) != 1 || min_size < 1) {
    stop("min_size must be a single number >= 1.")
  }
```

```r
  K <- as.integer(K)
  min_size <- as.integer(min_size)

  cluster_size <- switch(
    dist,
    poisson = {
      if (is.null(lambda) || lambda <= 0) stop("lambda must be > 0 for poisson.")
      stats::rpois(K, lambda)
    },
    nbinom = {
      if (is.null(size) || size <= 0) stop("size must be > 0 for nbinom.")
      if (is.null(prob) || prob <= 0 || prob >= 1) stop("prob must be in (0, 1) for nbinom.")
      stats::rnbinom(K, size = size, prob = prob)
    },
    uniform = {
      if (is.null(max_size) || max_size < min_size) {
        stop("max_size must be >= min_size for uniform.")
      }
      sample(min_size:max_size, K, replace = TRUE)
    },
    fixed = {
      if (is.null(fixed_size) || fixed_size < 1) {
        stop("fixed_size must be >= 1 for fixed.")
      }
      rep(as.integer(fixed_size), K)
    }
  )

  keep <- which(cluster_size >= min_size)

  if (length(keep) == 0) {
    stop("No clusters satisfy the minimum size condition.")
  }

  kept_sizes <- cluster_size[keep]

  dat.cluster <- data.frame(
    cluster_id = rep(seq_along(keep), times = kept_sizes)
  )

  dat.cluster$subject_id <- seq_len(nrow(dat.cluster))
  dat.cluster$interference_set_size <- kept_sizes[dat.cluster$cluster_id] - 1

  dat.cluster <- dat.cluster[, c("subject_id", "cluster_id", "interference_set_size")]

  attr(dat.cluster, "cluster_sizes") <- kept_sizes
  attr(dat.cluster, "dist") <- dist
  attr(dat.cluster, "n_clusters_kept") <- length(keep)
  attr(dat.cluster, "original_K") <- K

  return(dat.cluster)
}
```

Simulate Cluster Outcome Function

```r
simulate_cluster_outcome <- function(dat.cluster,
                                     p_L = 0.4,
                                     outcome_intercept = 0.5,
                                     beta_a = -0.5,
                                     beta_abar = -0.5,
                                     beta_a_abar = 0.25,
                                     beta_l = 0.25,
                                     propensity_intercept = -0.5,
                                     propensity_L = 1,
                                     define_observed_outcome = FALSE) {

  # generate covariate
  dat.cluster$L_i <- rbinom(nrow(dat.cluster), 1, p_L)

  data <- dat.cluster

  n <- nrow(data)

  Ni <- data$interference_set_size
  Li <- data$L_i

  PO <- vector("list", length = n)

  for (i in seq_len(n)) {
    PO[[i]] <- matrix(NA_integer_, nrow = 2, ncol = Ni[i] + 1)
  }

  for (i in seq_len(n)) {
    for (j in 0:1) {
      for (k in 0:Ni[i]) {

        abar <- if (Ni[i] == 0) 0 else k / Ni[i]

        eta <- outcome_intercept +
          beta_a * j +
          beta_abar * abar +
          beta_a_abar * j * abar +
          beta_l * Li[i]

        p <- plogis(eta)

        PO[[i]][j + 1, k + 1] <- rbinom(1, 1, p)
      }
    }
  }

  if (!define_observed_outcome) {
    return(PO)
  }

  # treatment assignment
```

```r
  n_cl <- nrow(dat.cluster)

  dat.cluster$Ai <- rbinom(
    n_cl,
    size = 1,
    prob = plogis(propensity_intercept + propensity_L * dat.cluster$L_i)
  )

  # cluster neighbor treatment exposure
  dat.cluster <- dat.cluster %>%
    dplyr::group_by(cluster_id) %>%
    dplyr::mutate(
      treated_neighbors = sum(Ai) - Ai,
      Abar_Ni = ifelse(interference_set_size == 0,
                       0,
                       treated_neighbors / interference_set_size),
      n = dplyr::n()
    ) %>%
    dplyr::ungroup()

  # observed outcome
  dat.cluster$Y_obs <- vapply(
    seq_len(nrow(dat.cluster)),
    function(i) {
      PO_i <- PO[[i]]
      PO_i[
        dat.cluster$Ai[i] + 1,
        dat.cluster$treated_neighbors[i] + 1
      ]
    },
    numeric(1)
  )

  return(dat.cluster)
}
```

Construct clustered data

```r
# Build clustered structure
dat.cluster <- build_cluster(
  K = 1000,
  dist = "poisson",
  lambda = 10,
  min_size = 2,
  seed = 123
)

head(dat.cluster)
```

```
##   subject_id cluster_id interference_set_size
## 1          1          1                     7
```

```
## 2          2          1                      7
## 3          3          1                      7
## 4          4          1                      7
## 5          5          1                      7
## 6          6          1                      7
```

Description

The generated dataset includes the following variables:

- `subject_id`: a unique identifier for each individual
- `cluster_id`: the cluster to which each individual belongs
- `interference_set_size`: the number of other individuals in the same cluster (excluding the individual itself)

For each individual i, the interference set is defined as all other members within the same cluster.

Generate potential outcomes

```r
PO <- simulate_cluster_outcome(
  dat.cluster,
  p_L = 0.4,
  outcome_intercept = 0.5,
  beta_a = -0.5,
  beta_abar = -0.5,
  beta_a_abar = 0.25,
  beta_l = 0.25,
  propensity_intercept = -0.5,
  propensity_L = 1,
  define_observed_outcome = FALSE
)

length(PO)
```

```
## [1] 9983
```

```r
PO[[1]]
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8]
## [1,]    1    1    1    0    1    0    1    0
## [2,]    1    1    0    0    0    0    0    0
```

Description

For each individual i, the function generates a matrix of potential outcomes.

- Rows correspond to individual treatment values $a \in \{0, 1\}$

- Columns correspond to the number of treated neighbors $k \in \{0, \ldots, N_i\}$

Thus, the entry

$$PO[[i]][a+1, k+1]$$

represents the potential outcome for individual i under treatment a and exactly k treated neighbors in the interference set.

The outcome probability is generated from the logistic model

$$\text{logit}\{P(Y_i = 1)\} = \beta_0 + \beta_a a + \beta_{\bar{a}} \bar{a} + \beta_{a\bar{a}}(a \times \bar{a}) + \beta_l L_i,$$

where

$$\bar{a} = \frac{k}{N_i}.$$

Here, L_i is a binary baseline covariate, β_0 is the outcome intercept, β_a is the main effect of individual treatment, $\beta_{\bar{a}}$ is the main effect of neighborhood treatment exposure, $\beta_{a\bar{a}}$ is the interaction between individual treatment and neighborhood exposure, and β_l is the effect of the baseline covariate.

Define observed treatment and observed outcome

```r
dat.obs <- simulate_cluster_outcome(
  dat.cluster,
  p_L = 0.4,
  outcome_intercept = 0.5,
  beta_a = -0.5,
  beta_abar = -0.5,
  beta_a_abar = 0.25,
  beta_l = 0.25,
  propensity_intercept = -0.5,
  propensity_L = 1,
  define_observed_outcome = TRUE
)

head(dat.obs)
```

```
## # A tibble: 6 x 9
##   subject_id cluster_id interference_set_size   L_i    Ai treated_neighbors
##        <int>      <int>                 <dbl> <int> <int>             <int>
## 1          1          1                     7     0     0                 3
## 2          2          1                     7     0     0                 3
## 3          3          1                     7     0     0                 3
## 4          4          1                     7     0     0                 3
## 5          5          1                     7     1     1                 2
## 6          6          1                     7     1     1                 2
## # i 3 more variables: Abar_Ni <dbl>, n <int>, Y_obs <dbl>
```

Description

When `define_observed_outcome = TRUE`, the function additionally generates observed treatment, neighborhood exposure, and observed outcomes.

Individual treatment is assigned according to the propensity score model

$$P(A_i = 1 \mid L_i) = \text{expit}(\alpha_0 + \alpha_1 L_i),$$

where α_0 corresponds to `propensity_intercept` and α_1 corresponds to `propensity_L`.

For each individual i, the number of treated neighbors is computed as

$$\text{treated_neighbors}_i = \sum_{j \in C(i)} A_j - A_i,$$

where $C(i)$ denotes the cluster containing individual i.

The proportion of treated neighbors is then defined as

$$\bar{A}_{N_i} = \begin{cases} 0, & N_i = 0, \\ \dfrac{\text{treated_neighbors}_i}{N_i}, & N_i > 0. \end{cases}$$

The observed outcome is obtained by consistency:

$$Y_i^{\text{obs}} = Y_i\big(A_i, \text{treated_neighbors}_i\big).$$

Output

Potential outcome output

When `define_observed_outcome = FALSE`, the function returns a list of potential outcome matrices:

```r
class(PO)
```

```
## [1] "list"
```

```r
length(PO)
```

```
## [1] 9983
```

Observed data output

When `define_observed_outcome = TRUE`, the function returns a data frame containing the observed data:

```r
colnames(dat.obs)
```

```
## [1] "subject_id"            "cluster_id"            "interference_set_size"
## [4] "L_i"                   "Ai"                    "treated_neighbors"
## [7] "Abar_Ni"               "n"                     "Y_obs"
```

The returned data frame includes:

- `subject_id`
- `cluster_id`
- `interference_set_size`
- `L_i`
- `Ai`
- `treated_neighbors`
- `Abar_Ni`
- `n`
- `Y_obs`

Example

```r
set.seed(123)

dat.cluster <- build_cluster(
  K = 1000,
  dist = "poisson",
  lambda = 10,
  min_size = 2
)

# potential outcomes only
PO <- simulate_cluster_outcome(
  dat.cluster,
  define_observed_outcome = FALSE
)

# observed data
obs.dat <- simulate_cluster_outcome(
  dat.cluster,
  define_observed_outcome = TRUE
)

head(obs.dat)
```

```
## # A tibble: 6 x 9
##   subject_id cluster_id interference_set_size   L_i    Ai treated_neighbors
##        <int>      <int>                 <dbl> <int> <int>             <int>
## 1          1          1                     7     0     0                 3
## 2          2          1                     7     0     0                 3
## 3          3          1                     7     0     0                 3
## 4          4          1                     7     0     0                 3
```

```
## 5          5          1                     7     1     1                  2
## 6          6          1                     7     1     1                  2
## # i 3 more variables: Abar_Ni <dbl>, n <int>, Y_obs <dbl>
```